Домашнее задание 14 — Введение в проект Loom (Virtual Threads)

- **Дедлайн:** 04.11.2026 (текст задания открылся позже, отчёт дополнен 09.11.2026)
- **Тема:** Введение в проект Loom (Virtual Threads)
- **Форма проверки:** задание с самопроверкой
- **Вариант итогового ДЗ:** 3 — парсер вакансий
- **Модуль:** `hw14.loom` в Maven-проекте `app/`
- **JDK:** 21 (Temurin 21.0.13), в `pom.xml` — `<java.version>21</java.version>` + `--enable-preview` (почему именно так — раздел 1.1)
- **Дата замеров:** 09.11.2026, Linux, 12 процессоров, 23 ГиБ ОЗУ

---

1. Требования задания и что сделано

Текст задания (Netology, дедлайн 04.11.2026) состоит из шести пунктов и требований к отчёту. Ниже — карта «требование → где сделано», дальше по разделам — как именно.

№	Требование (дословно по смыслу)	Где сделано
1	**Создание задач:** 10 000 задач; каждая выводит имя своего потока и имитирует неблокирующий IO (`Thread.sleep` / `Thread.yield`)	`swarm.ThreadSwarm` , `swarm.SwarmRun` — этап 5 демонстрации
2	**Виртуальные потоки:** выполнить все 10 000 задач через `Thread.ofVirtual().start(...)`	`ThreadSwarm.runVirtual` (этап 5) + `benchmark.LoomBenchmarks.runNamedVirtual` (этапы 1–2)
3	**Платформенные потоки:** те же задачи через `new Thread(...).start()` ; сравнить поведение и производительность	`ThreadSwarm.runPlatform` (этап 5), `LoomBenchmarks.runPlatform` (этапы 2, 4)
4	**Сравнение:** замерить время (`System.nanoTime()`), сравнить	`SwarmRun.startNanos/totalNanos/cpuNanos` , `BenchmarkResult.wallTimeNanos` ; таблицы в `report.LoomReport`
5	**StructuredTaskScope:** управление группой виртуальных потоков, ожидание завершения всех задач, обработка исключений и отмены	`structured.StructuredExperiments` , `structured.ScopeRun` — этап 6: четыре опыта (`join` , падение подзадачи, `joinUntil` + `shutdown` , `ShutdownOnSuccess`)
6	**Тестирование:** прогоны с разными количествами задач, сравнение времени и потребления ресурсов (CPU/память)	параметры `app.hw14.*` , серия прогонов 1000/5000/10 000 (раздел 2.5.3), CPU (`OperatingSystemMXBean`) и куча (`MemoryProbe`) в каждом прогоне
—	**Отчёт:** этапы, промежуточные результаты (пример вывода имени потока, замеры), объяснение работы виртуальных потоков, `StructuredTaskScope` и обработки исключений, итоговые выводы	этот файл, разделы 2–5

Что уже было в модуле до открытия текста задания и осталось: загрузка 200 страниц вакансий с локального сайта-заглушки в виртуальных потоках (этап 1), сравнение с платформенным пулом (этап 2), опыт с pinning (этап 3), масштабирование на 10 000 задач (этап 4). Новое: этап 5 («поток на задачу» напрямую через `Thread.ofVirtual().start` и `new Thread().start`) и этап 6 (`StructuredTaskScope`), 13 новых тестов, этот отчёт и пересобранный PDF.

Предметная область та же, что в остальных ДЗ, — парсер вакансий. Страницы грузятся с локального сайта-заглушки на свободном порту, поэтому интернет не нужен, а задержка ответа делает разницу между исполнителями измеримой: виртуальные потоки масштабируются на **ожидании**, а не на процессоре.

1.1. Про версию JDK и `--enable-preview` (честно)

`StructuredTaskScope` живёт в `java.util.concurrent`, но **до сих пор остаётся preview-API**:

- JDK 21 — JEP 453, первое preview в `java.util.concurrent` (до этого — инкубатор `jdk.incubator.concurrent` в 19–20);
- JDK 25 — JEP 505 «Structured Concurrency (Fifth Preview)», **не финальный** API;
- JDK 27 — тоже preview.

Проверено на реальных JDK: `javac 25.0.5` и `javac 27.0.1` на строке `import java.util.concurrent.StructuredTaskScope;` отвечают `error: StructuredTaskScope is a preview API and is disabled by default`. Финальным API не стал ни в одной из доступных версий, а инкубаторного модуля `jdk.incubator.concurrent` в JDK 21 уже нет (`java --list-modules` его не показывает). Поэтому вариантов два: включать preview или не использовать `StructuredTaskScope` вообще.

Выбран **JDK 21 + `--enable-preview`** по двум причинам:

1. **Pinning ещё работает.** В JDK 24+ (JEP 491) `synchronized` больше не пришпиливает виртуальные потоки, и этап 3 теряет смысл: прогоны становятся одинаковыми. Проверено на JDK 25: из 19 прежних тестов падают два (`PinningExperimentTest` — пик 24 вместо «не больше 12»). С JDK 21 все прежние выводы и замеры остаются в силе.
2. **API JDK 21 — тот самый, о котором говорит текст задания**: `joinUntil(deadline)`, `shutdown()`, `ShutdownOnFailure` / `ShutdownOnSuccess`, `throwIfFailed()`. В JDK 25 API переработан (`open(Joiner)`, `Configuration.withTimeout`), и отчёт пришлось бы писать про другой класс.

Цена решения: собирать и запускать надо с флагом `--enable-preview` (он уже прописан в `pom.xml` для компилятора и surefire), а готовый jar работает только на JDK 21 — preview-классы помечены версией `65.65535` и другая JVM их не загрузит. Это ограничение указано и в разделе 9.

Остальные 100 % кода (всё, кроме `structured.*`) — обычный Java 21 без preview.

2. Этапы выполнения

2.1. Этап 1. Виртуальные потоки базово

```
 ThreadFactory factory = Thread.ofVirtual().name("hw14-vt-", 0).factory();
try (ExecutorService executor = Executors.newThreadPerTaskExecutor(factory)) {
    // на каждую страницу — своя задача; внутри блокирующий client.send(...)
}
```

`PageVacancyClient.fetch` намеренно **синхронный** (`HttpClient.send`, не `sendAsync`): именно блокирующий вызов позволяет виртуальному потоку сняться с carrier-потока и отдать его другим задачам. С асинхронным клиентом ждать было бы нечего и выигрыш исчез.

Каждый результат несёт «паспорт» потока (`ThreadProbe`): виртуальный ли он и на каком carrier смонтирован. Журнал демонстрации:

```
 виртуальные потоки: задач 200 (вернули результат 200, исключений 0, отменено 0), 159.1 мс,
                   пик одновременных 200, потоков-исполнителей 200, прирост потоков JVM 2
без потерь: да, разных страниц 200 из 200, сбоев 0
примеры заголовков:
```

```
  №0 «Вакансия №0 — Java-разработчик» (virtual@ForkJoinPool-1-worker-2)
  №1 «Вакансия №1 — Backend-разработчик (Java)» (virtual@ForkJoinPool-1-worker-9)
  №2 «Вакансия №2 — Разработчик Spring Boot» (virtual@ForkJoinPool-1-worker-8)
паспорта потоков (первые 3):
  №0 поток hw14-vt-0, виртуальный: да, carrier: ForkJoinPool-1-worker-2
  №1 поток hw14-vt-1, виртуальный: да, carrier: ForkJoinPool-1-worker-9
  №2 поток hw14-vt-2, виртуальный: да, carrier: ForkJoinPool-1-worker-8
```

Наблюдения:

- 200 задач выполнились за 159.1 мс при задержке страницы 50 мс — то есть почти все ждали одновременно, а не по очереди;
- разные задачи попали на разные carrier-потоки (всего их 12 — по числу процессоров), при этом «потоков-исполнителей 200»: виртуальных потоков двести, потоков ОС — двенадцать;
- прирост счётчика платформенных потоков JVM — 2 (служебные потоки клиента), а не 200: виртуальные потоки в `ThreadMXBean` не входят;
- потерь нет: 200 разных страниц, 200 успешных загрузок, сбоев 0.

Перед замером выполняется прогрев на 32 страницах (в замеры не входит): первый HTTP-запрос подгружает классы клиента и искажает «холодное» число.

2.2. Этап 2. Тот же объём работы: платформенный пул и виртуальные потоки

Один и тот же список из 200 адресов, один и тот же клиент, один и тот же код задачи. Меняется только исполнитель: `Executors.newFixedThreadPool(16)` и `Executors.newVirtualThreadPerTaskExecutor()`.

Исполнитель	Время, мс	Стр./с	Пик задач	Потоков-исполнителей	Исполнений virt/platf	Прирост потоков JVM
Платформенный пул (16)	699.5	285.9	16	16	0/200	+16
Виртуальные потоки	103.1	1939.9	200	200	200/0	0

Вывод журнала: «тот же объём работы, виртуальные потоки быстрее в **6.8 раза** (103.1 мс против 699.5 мс) — выигрыш даёт ожидание IO, а не процессор».

Почему пул проигрывает ровно во столько раз, сколько потоков ему не хватает: 200 страниц × 50 мс / 16 потоков ≈ 625 мс — это нижняя граница для пула, и замеренные 700 мс к ней близки (остальное — накладные расходы). Виртуальные потоки не ждут очереди: все 200 запросов стартуют сразу, и время определяется одной задержкой (50 мс) плюс накладные расходы на 200 соединений.

Обратите внимание на две последние колонки: у пула 200 исполнений на платформенных потоках и +16 потоков ОС, у виртуальных — 200 исполнений на виртуальных потоках и **+0** потоков ОС. Это и есть ответ на вопрос «в чём разница»: количество работы то же, количество потоков ОС — нет.

2.3. Этап 3. Pinning: `synchronized` против `ReentrantLock`

Что такое pinning. Виртуальный поток снимается с carrier-потока на время блокировки — на этом и держится масштабирование. Но сняться можно не всегда: внутри `synchronized` поток удерживает монитор, и планировщик не может «заморозить» его на полпути. Такой поток **пришпиливается** (pinned) к carrier-потоку и блокирует его на всё время ожидания. `ReentrantLock` — очередь ожидания на уровне JDK, а не монитор JVM, поэтому с ним виртуальный поток снимается штатно.

Как поставлен опыт. У каждой задачи **свой** монитор (или свой `ReentrantLock`), поэтому состязания за общий ресурс нет — единственная разница между прогонами в том, пришпилен поток или нет. Внутри секции задача ждёт 60 мс. 96 задач, 12 carrier-потоков:

Защита	Время, мс	Пик задач	Carrier'ов	Смен carrier'а	Ожидание при pinning, мс
`synchronized`	543.6	12	12 из 12	0	480.0
`ReentrantLock`	61.1	96	10 из 12	87	480.0

Вывод журнала: «`synchronized` удерживает поток на carrier'е — одновременно работают не больше 12 задач, прогон идёт 543.6 мс; `ReentrantLock` отпускает carrier — одновременно работают все 96 задач, прогон идёт 61.1 мс».

Три независимых признака одного и того же явления:

1. **Пик одновременных задач.** При `synchronized` — 12 (по числу carrier-потоков), при `ReentrantLock` — 96 (все задачи).
2. **Время.** 96 задач по 60 мс в 12 «кругов» — это 8 × 60 = 480 мс; замеренные 543.6 мс немного больше (первый прогон после разгона JIT), в следующих — 484 мс. Свободные потоки проходят те же 96 задач за одну блокировку — 61 мс.
3. **Смена carrier'а.** Пришпиленный поток не мог сменить carrier — 0 смен. Свободный поток после сна просыпается на любом свободном carrier'е — 87 смен из 96 задач.

Это не «медленный synchronized»: сам монитор здесь не горячий, конкуренции нет. Разница только в том, что один механизм блокирует поток ОС, а второй — нет.

> На JDK 24+ этот опыт даст другие числа (оба прогона ~60 мс): JEP 491 отменил пришпиливание внутри `synchronized`. Модуль поэтому собран и запускается на JDK 21 — см. раздел 1.1.

2.4. Этап 4. Масштабирование: 10 000 задач с ожиданием

Исполнитель	Задач	Время, мс	Задач/с	Пик задач	Потоков-исполнителей	Прирост потоков JVM
Платформенный пул (200)	10 000	2512.6	3979.9	200	200	+187
Виртуальные потоки	10 000	75.2	133 049.6	10 000	10 000	0

Вывод журнала: «10 000 задач с ожиданием — виртуальные потоки быстрее в **33.4 раза**; платформенный пул на 200 потоков ограничен их числом, виртуальные — нет».

Расчёт для пула: 10 000 задач / 200 потоков = 50 порций × 50 мс = 2500 мс — совпадает с замеренными 2513 мс. Виртуальные потоки стартуют все сразу и ждут параллельно: 75 мс — это одна задержка плюс накладные расходы планировщика. При этом потоков ОС по-прежнему около двенадцати (прирост счётчика JVM — 0), а «пик задач» — 10 000: одновременно ждут десять тысяч виртуальных потоков, а не десять тысяч потоков ОС.

2.5. Этап 5. «Поток на задачу»: `Thread.ofVirtual().start()` против `new Thread().start()`

Пункты 1–4 задания буквально: 10 000 задач, каждая **печатает имя своего потока** и имитирует неблокирующий IO (`Thread.sleep(1)`), запуск — напрямую, без пулов и executor'ов.

2.5.1. Код

Виртуальные потоки:

```
 Thread.Builder.OfVirtual builder = Thread.ofVirtual().name("hw14-swarm-vt-", 0);
for (int index = 0; index < tasks; index++) {
    int taskNumber = index;
    threads.add(builder.start(() -> task(taskNumber, names, 1L, ...)));
```

```
}
for (Thread thread : threads) {
    thread.join();          // ожидание всех задач
}
```

Платформенные потоки — то же самое, но `new Thread(...)`:

```
 Thread thread = new Thread(runnable, "hw14-swarm-pt-" + (index + 1));
thread.start();
```

Тело задачи одинаковое:

```
 Thread current = Thread.currentThread();
names[index] = current.getName();   // «каждая печатает имя своего потока»
if (current.isVirtual()) {
    virtualExecutions.incrementAndGet();
} else {
    platformExecutions.incrementAndGet();
}
echo.accept(current.getName());     // вывод в журнал (первые N строк)
Thread.sleep(sleepMillis);          // имитация неблокирующего IO
```

Имя каждой из 10 000 задач сохраняется в массив, поэтому проверяется не «на глаз», а тестом: имён ровно столько же, сколько задач, и они уникальны (у платформенных — `hw14-swarm-pt-1…10000`). В консоль печатаются первые 8 строк каждого прогона (иначе отчёт заливают 20 000 строк):

```
  1) поток hw14-swarm-vt-1
 2) поток hw14-swarm-vt-11
 ...
 8) поток hw14-swarm-vt-7
 1) поток hw14-swarm-pt-1
 ...
 8) поток hw14-swarm-pt-10
```

2.5.2. Результат

Исполнитель	Задач	Старт, мс	Всего, мс	CPU процесса, мс	Задач/с	Пик задач	Разных имён
`Thread.ofVirtual().start`	10 000	48.5	65.3	520	153 160	7983	10 000
`new Thread(...).start`	10 000	424.3	425.4	1290	23 507	49	10 000

Вывод журнала: «те же 10 000 задач — виртуальные потоки в **6.5 раза** быстрее (65.3 мс против 425.4 мс), но главное не время, а цена потока: запуск 10 000 потоков ОС занял 424.3 мс, запуск 10 000 виртуальных — 48.5 мс; пик одновременных задач 7983 против 49».

Как это читать:

- **Время почти целиком уходит на создание потоков.** У платформенных `start()` — 424.3 мс из 425.4 мс: сама задача (сон 1 мс) не стоит ничего. У виртуальных `start()` — 48.5 мс, остальные ~17 мс — ожидание задач и работа планировщика. Отсюда и разница в 6.5 раза: 10 000 потоков ОС — это 10 000 стеков (по умолчанию 1 МиБ виртуальной памяти, то есть ~10 ГиБ адресного

пространства), 10 000 записей в планировщике и структур ядра. Виртуальный поток — объект в куче, стек растёт по мере надобности.

- **Пик одновременных задач у платформенных — 49, а не 10 000.** Это не ошибка замера: пауза всего 1 мс, JVM создаёт ~23 потока в миллисекунду, и столько же успевает завершиться. Потоки ОС физически не накапливаются — тем показательнее, что даже без накопления запуск стоит почти полсекунды. У виртуальных пик 7983: почти все 10 000 действительно живы одновременно (остальные успели закончиться, пока стартовали следующие).
- **CPU процесса**: 1290 мс против 520 мс на те же 10 000 задач — платформенные потоки съедают в 2.5 раза больше процессорного времени на создание/уничтожение и переключения.
- **Куча** (оценка без `System.gc()`): 22.6 МиБ у виртуальных против 0.0 МиБ у платформенных — и это ровно тот случай, когда «меньше по куче» не значит «дешевле»: стеки потоков ОС живут в нативной памяти и в `Runtime.totalMemory()` не видны. Цена платформенных потоков — вне кучи.
- **Потоков JVM +0 в обоих прогонах** — счётчик `ThreadMXBean` хранит исторический пик на процесс, и в общем прогоне демонстрации он уже поднимался выше. Поэтому главный показатель — «пик задач» из счётчика внутри задач и «имён» = 10 000.

Если машина не даёт 10 000 потоков ОС (низкий `ulimit -u`, мало памяти), запуск можно разбить на пачки: `--app.hw14.swarm-max-platform-threads=N`. Тогда `ThreadSwarm` стартует по N потоков, ждёт пачку, стартует следующую, а в отчёте это видно по колонке «пачек» и по метке в названии прогона. При совсем жёстком отказе ядра (`OutOfMemoryError: unable to create native thread`) прогон автоматически повторяется пачками помельче — демонстрация не должна падать из-за настроек системы.

2.5.3. Разные количества задач (пункт 6 задания)

Те же задачи с `--app.hw14.swarm-tasks=1000 / 5000 / 10000` (остальные этапы в этих прогонах тоже выполнялись, здесь приведены только строки этапа 5):

Задач	Виртуальные: всего, мс	из них старт, мс	Платформенные: всего, мс	из них старт, мс	Пик задач (vt / pt)	CPU процесса, мс (vt / pt)
1 000	4.6	2.9	84.6	83.5	980 / 46	30 / 420
5 000	33.2	7.5	286.9	285.8	3438 / 47	280 / 980
10 000	54.5	39.0	456.4	455.3	6057 / 49	440 / 1420

Видно, что рост линейный по числу потоков и у платформенных он в ~10 раз дороже по времени и в 3–5 раз по CPU: `new Thread().start()` — это системный вызов и выделение стека, а `Thread.ofVirtual().start()` — создание объекта и постановка в очередь планировщика.

2.6. Этап 6. `StructuredTaskScope`: группа виртуальных потоков

2.6.1. Что это и зачем

Structured concurrency — это дисциплина, в которой параллельные подзадачи не «висят сами по себе», а принадлежат блоку кода: задача разбивается на подзадачи, и пока блок не закрыт, наружу выйти нельзя. `StructuredTaskScope` — реализация этого в JDK: скоуп открывается в `try-with-resources`, подзадачи форкаются, владелец ждёт их, а `close()` гарантирует, что ни одна подзадача не осталась работать.

Три свойства, которых нет у «сырых» потоков и у пулов:

1. **Жизненный цикл ограничен блоком.** После выхода из `try` подзадач нет. Нельзя «забыть» поток: если владелец форкнул и не вызвал `join`, `close()` бросит `IllegalStateException`.
2. **Отмена распространяется.** `shutdown()` прерывает незавершённые подзадачи, а `close()` дожидается их фактического завершения (поэтому важно, чтобы подзадача реагировала на прерывание — `Thread.sleep`, чтение из сокета, `Future.get` реакцию поддерживают).

3. **Ошибки собираются в одном месте.** Политика (`ShutdownOnFailure`, `ShutdownOnSuccess`) решает, что считать успехом группы; вызывающий код получает один итог, а не десяток `Future`.

API JDK 21, которое здесь используется: `fork(Callable)` → `Subtask`, `join()`, `joinUntil(Instant)`, `shutdown()`, `throwIfFailed()`, `Subtask.get()/exception()/state()`, политики `ShutdownOnFailure` (первый сбой отменяет остальных) и `ShutdownOnSuccess` (первый успех отменяет остальных).

2.6.2. Опыт 1: все подзадачи успешны

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure(
        "hw14-sts-all", Thread.ofVirtual().name("hw14-sts-", 0).factory())) {
    for (int index = 0; index < tasks; index++) {
        scope.fork(() -> loadVacancy(index));      // 10 000 подзадач
    }
    scope.join();          // ждём завершения ВСЕХ подзадач
    scope.throwIfFailed(); // если кто-то упал — ExecutionException с причиной
}                          // close(): подзадач не осталось
```

```
1) join() ждёт всю группу — все успешны (ShutdownOnFailure): подзадач 10000,
   с результатом 10000, с исключением 0, отменено 0, 319.2 мс,
   пик одновременных 10000, итог: успех
   результатов после ожидания: 10000, все успешны: да, все подзадачи виртуальные: да
```

319 мс при задержке 300 мс — это «одна волна»: все 10 000 подзадач стартовали и ждали одновременно, пик — ровно 10 000.

2.6.3. Опыт 2: одна подзадача падает — остальные отменяются

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure(...)) {
    for (int index = 0; index < tasks; index++) {
        scope.fork(() -> {
            if (index == failingTask) {
                Thread.sleep(failAfterMillis);
                throw new IllegalStateException("источник недоступен: " + index);
            }
            Thread.sleep(delayMillis);
            return index;
        });
    }
    scope.join();          // вернётся по первому сбою: ShutdownOnFailure вызвал shutdown()
    scope.throwIfFailed(); // ExecutionException с исходной причиной
}
```

```
2) одна падает (ShutdownOnFailure): подзадач 10000, с результатом 0, с исключением 1,
   отменено 9999, 137.9 мс, пик одновременных 10000, итог: исключение
```

Что произошло по шагам:

1. 10 000 подзадач форкнуты и спят по 300 мс; подзадача №137 через 100 мс бросает `IllegalStateException`.
2. `ShutdownOnFailure` видит сбой и вызывает `shutdown()` — остальные 9 999 подзадач получают `InterruptedException` и завершаются, не доработав свои 300 мс.

3. `join()` возвращается, `throwIfFailed()` бросает `ExecutionException` с причиной `IllegalStateException: источник недоступен: подзадача №137`.
4. `close()` ждёт, пока все прерванные потоки реально закончат.

Без отмены группа шла бы не меньше 400 мс (100 мс до сбоя + 300 мс паузы остальных), с отменой — 138 мс. Счётчики сходятся: 0 с результатом + 1 с исключением + 9999 отменённых = 10 000.

2.6.4. Опыт 3: срок истёк — отмена по таймауту

```
 Instant deadline = Instant.now().plusMillis(timeoutMillis);
try (var scope = new StructuredTaskScope.ShutdownOnFailure(...)) {
    for (int index = 0; index < tasks; index++) {
        scope.fork(() -> loadVacancySlow(index));   // спят по 5 с
    }
    try {
        scope.joinUntil(deadline);   // ждём не дольше срока
        scope.throwIfFailed();
    } catch (TimeoutException e) {
        scope.shutdown();            // отменяем незавершённые; close() дождётся
    }
}
```

```
3) таймаут (ShutdownOnFailure): подзадач 10000, с результатом 0, с исключением 0,
   отменено 10000, 268.9 мс, пик одновременных 10000, итог: таймаут
```

`joinUntil` сам **не** останавливает подзадачи — он только перестаёт ждать и бросает `TimeoutException`. Отмена — явная: `shutdown()` прерывает все незавершённые, `close()` ждёт их выхода. 269 мс при сроке 250 мс — это срок плюс стоимость прерывания 10 000 потоков.

2.6.5. Опыт 4: первый успешный ответ отменяет дубли

```
try (var scope = new StructuredTaskScope.ShutdownOnSuccess<String>(
        "hw14-sts-first", Thread.ofVirtual().name("hw14-sts-", 0).factory())) {
    for (long delay : List.of(400L, 150L, 250L)) {
        scope.fork(() -> loadFromMirror(delay));   // три «зеркала»
    }
    scope.join();
    return scope.result();      // первый успех; остальные уже отменены
}
```

```
4) первый успешный источник — первый успех (ShutdownOnSuccess): подзадач 3,
   с результатом 1, с исключением 0, отменено 2, 152.8 мс, пик одновременных 3,
   итог: успех, ответ: источник №1 (ответ через 150 мс)
```

Приём для реального парсера: пока ждём самый медленный источник (400 мс), быстрый отвечает за 150 мс, и два оставшихся запроса отменяются — ровно та же схема, что «retry по зеркалам», только короче и без утечек.

2.6.6. Сводка и обработка исключений

Сценарий	Политика	Подзадач	Время, мс	С результатом	С исключением	Отменено	Итог
Все подзадачи	`ShutdownOnFailure`	10 000	319.2	10 000	0	0	успех

Сценарий	Политика	Подзадач	Время, мс	С результатом	С исключением	Отменено	Итог
успешны							
Одна падает	`ShutdownOnFailure`	10 000	137.9	0	1	9999	исключение
Таймаут (250 мс)	`ShutdownOnFailure`	10 000	268.9	0	0	10 000	таймаут
Первый успех	`ShutdownOnSuccess`	3	152.8	1	0	2	успех

Обработка исключений и отмены — что именно делает код:

- **Сбой подзадачи не теряется.** `Callable` подзадачи бросает исключение → оно попадает в `Subtask.exception()`, `ShutdownOnFailure.exception()` отдаёт первое, `throwIfFailed()` заворачивает его в `ExecutionException`; в коде причина разворачивается `rootCause(...)`, поэтому в отчёте видно `IllegalStateException: источник недоступен: подзадача №137`, а не безымянный `ExecutionException`.
- **Отмена — это `interrupt`.** `shutdown()` прерывает потоки подзадач; те, что написаны правильно (`Thread.sleep`, блокирующий ввод-вывод), сразу выходят. Подзадача, которая глотает прерывание, задержит `close()` — поэтому «не глотать `InterruptedException`» здесь не стилистика, а условие работы механизма.
- **Состояния считаются, а не предполагаются.** Каждая подзадача пишет своё состояние (`SUCCESS` / `FAILED` / `CANCELLED`), после `join()` / `close()` владелец суммирует их; инвариант «успех + сбой + отмена = число подзадач» проверяется тестами (`ScopeRun.settled()`).
- **Таймаут не равен отмене.** `joinUntil` только будит владельца; отменять нужно явно (`shutdown()`), иначе `close()` будет ждать подзадачи до конца.

3. Фактический вывод демонстрации

```
 cd app
mvn -DskipTests package
java --enable-preview -jar target/vacancy-parser.jar --app.hw=hw14 --server.port=0
```

Прогон завершился с кодом `0`, живых платформенных потоков ДЗ 14 после завершения нет. Ниже — вывод целиком; длинные строки консоли перенесены для читаемости (в терминале они идут одной строкой). Таблицы в журнале выровнены моноширинным шрифтом терминала; в PDF кириллица набрана другим начертанием, поэтому столбцы могут слегка «плыть» — сводные таблицы с теми же числами есть в разделе 2.

```
 === ДЗ 14. Проект Loom: виртуальные потоки на блокирующем IO ===


Процесс: pid 368520
Настройки ДЗ 14:
    - страниц: 200, задержка страницы: 50 мс
    - платформенный пул для сравнения: 16 потоков
    - масштабирование: 10000 задач по 50 мс, пул 200
    - pinning: 96 задач по 60 мс
    - поток на задачу: 10000 задач по 1 мс, предел платформенных потоков без ограничения
    - StructuredTaskScope: 10000 подзадач по 300 мс, падение через 100 мс, срок 250 мс
Планировщик виртуальных потоков: carrier-потоков (parallelism) 12, предел пула
```

```
      планировщика 256, процессоров 12

Этап 1. Виртуальные потоки: загрузка страниц вакансий
    источник: локальный сайт, http://127.0.0.1:37557, страниц 200
    исполнитель: Thread.ofVirtual().name("hw14-vt-", 0) + newThreadPerTaskExecutor
          (по виртуальному потоку на страницу)
    пример адреса: http://127.0.0.1:37557/vacancy/0
    прогрев: 32 страниц (в замеры не входят — первый запрос подгружает классы
          HTTP-клиента)
    виртуальные потоки: задач 200 (вернули результат 200, исключений 0, отменено 0),
          159.1 мс, 1257.0 задач/с, пик одновременных 200, потоков-исполнителей 200,
          прирост потоков JVM 2
    без потерь: да, разных страниц 200 из 200, сбоев 0
    примеры заголовков:
      №0 «Вакансия №0 — Java-разработчик» (virtual@ForkJoinPool-1-worker-2)
      №1 «Вакансия №1 — Backend-разработчик (Java)» (virtual@ForkJoinPool-1-worker-9)
      №2 «Вакансия №2 — Разработчик Spring Boot» (virtual@ForkJoinPool-1-worker-8)
      №3 «Вакансия №3 — Инженер по тестированию» (virtual@ForkJoinPool-1-worker-12)
      №4 «Вакансия №4 — Аналитик данных» (virtual@ForkJoinPool-1-worker-6)
    проверка: исполнений на виртуальных потоках 200, на платформенных 0 → все
          запросы сделаны виртуальными потоками: да
    паспорта потоков (первые 3):
      №0 поток hw14-vt-0, виртуальный: да, carrier: ForkJoinPool-1-worker-2
      №1 поток hw14-vt-1, виртуальный: да, carrier: ForkJoinPool-1-worker-9
      №2 поток hw14-vt-2, виртуальный: да, carrier: ForkJoinPool-1-worker-8

Этап 2. Сравнение с платформенным пулом: тот же объём работы
    сначала newFixedThreadPool(16), затем Executors.newVirtualThreadPerTaskExecutor()
    исполнитель         время, мс  стр./с  пик  потоков  virt/platf  JVM-потоков
    ------------------  ---------  ------  ---  -------  ----------  -----------
    платформенный пул   699.5      285.9   16   16       0/200       16
    виртуальные потоки  103.1      1939.9  200  200      200/0       0
    вывод: тот же объём работы, виртуальные потоки быстрее в 6.8 раза (103.1 мс
          против 699.5 мс) — выигрыш даёт ожидание IO, а не процессор
    целостность: платформенный пул — без потерь да, задач с результатом 200 из 200,
          страниц со сбоем 0; виртуальные потоки — без потерь да, задач с результатом 200
          из 200, страниц со сбоем 0
    память (приблизительно, без System.gc()): пул 8.9 МиБ, виртуальные 9.8 МиБ

Этап 3. Pinning: synchronized против ReentrantLock
    у каждой задачи свой монитор/замок, внутри секции — ожидание 60 мс
    защита         время, мс  пик задач  carrier'ов  смен carrier'а  ожидание, мс
    -------------  ---------  ---------  ----------  --------------  ------------
    synchronized   543.6      12         12 из 12    0               480.0
    ReentrantLock  61.1       96         10 из 12    87              480.0
    вывод: synchronized удерживает поток на carrier'е — одновременно работают не
          больше 12 задач, прогон идёт 543.6 мс; ReentrantLock отпускает carrier —
          одновременно работают все 96 задач и прогон идёт 61.1 мс
    задачи завершены: synchronized 96 из 96, ReentrantLock 96 из 96
    как это читать: при pinning виртуальный поток не отдаёт carrier, поэтому
          одновременно ждут не больше 12 задач; ReentrantLock позволяет снять поток с
          carrier и ждать всем сразу
```

```
Этап 4. Масштабирование: 10000 задач с ожиданием 50 мс
    исполнитель            задач  время, мс  задач/с   пик    рабочих  JVM-потоков
    ---------------------  -----  ---------  --------  -----  -------  -----------
    платформенный пул 200  10000  2512.6     3979.9    200    200      187
    виртуальные потоки     10000  75.2       133049.6  10000  10000    0
    вывод: 10000 задач с ожиданием — виртуальные потоки быстрее в 33.4 раза;
          платформенный пул на 200 потоков ограничен их числом, виртуальные — нет
    завершено задач: пул 10000 из 10000, виртуальные 10000 из 10000
    память (приблизительно): пул 9.7 МиБ, виртуальные 0.0 МиБ (GC отработал во время
          прогона — замер приблизительный)

Этап 5. 10000 задач с печатью имени потока: Thread.ofVirtual().start() против new
      Thread().start()
    каждая задача пишет имя своего потока и ждёт 1 мс (имитация неблокирующего IO)
      1) поток hw14-swarm-vt-0
      2) поток hw14-swarm-vt-11
      3) поток hw14-swarm-vt-10
      4) поток hw14-swarm-vt-9
      5) поток hw14-swarm-vt-8
      6) поток hw14-swarm-vt-7
      7) поток hw14-swarm-vt-6
      8) поток hw14-swarm-vt-5
    Thread.ofVirtual().start: задач 10000, завершено 10000, прервано 0, запуск 48.5
          мс, всего 65.3 мс, 153160 задач/с, пачек 1, пик одновременных 7983, разных имён
          10000, прирост потоков JVM 0, CPU процесса 520 мс
    имён собрано 10000, первых: hw14-swarm-vt-0, hw14-swarm-vt-1, hw14-swarm-vt-2
      1) поток hw14-swarm-pt-1
      2) поток hw14-swarm-pt-4
      3) поток hw14-swarm-pt-5
      4) поток hw14-swarm-pt-3
      5) поток hw14-swarm-pt-2
      6) поток hw14-swarm-pt-10
      7) поток hw14-swarm-pt-12
      8) поток hw14-swarm-pt-11
    new Thread(...).start: задач 10000, завершено 10000, прервано 0, запуск 424.3
          мс, всего 425.4 мс, 23507 задач/с, пачек 1, пик одновременных 49, разных имён
          10000, прирост потоков JVM 0, CPU процесса 1290 мс
    имён собрано 10000, первых: hw14-swarm-pt-1, hw14-swarm-pt-2, hw14-swarm-pt-3
    исполнитель               задач  старт, мс  всего, мс  CPU, мс  задач/с  пик   имён
    ------------------------  -----  ---------  ---------  -------  -------  ----  -----
    Thread.ofVirtual().start  10000  48.5       65.3       520      153160   7983  10000
    new Thread(...).start     10000  424.3      425.4      1290     23507    49    10000
    вывод: те же 10000 задач — виртуальные потоки в 6.5 раза быстрее (65.3 мс против
          425.4 мс), но главное не время, а цена потока: запуск 10000 потоков ОС занял
          424.3 мс, запуск 10000 виртуальных — 48.5 мс; пик одновременных задач 7983
          против 49
    целостность: виртуальные — завершено 10000 из 10000, разных имён 10000,
          исполнений на виртуальных 10000, на платформенных 0; платформенные — завершено
          10000 из 10000, разных имён 10000, исполнений на виртуальных 0, на платформенных
          10000
    память (приблизительно, без System.gc()): виртуальные 22.6 МиБ, платформенные
          0.0 МиБ (GC отработал во время прогона — замер приблизительный)
    цена: 10 000 потоков ОС — это ~10 ГиБ зарезервированного адресного пространства
```

```
        под стеки и десятки тысяч операций планировщика; виртуальные потоки — объекты в
        куче, стек растёт по мере надобности
    почему пик одновременных задач у платформенных меньше 10000: пауза всего 1 мс,
        потоки не успевают накопиться — JVM создаёт их примерно 24 в миллисекунду и
        столько же завершается; цена видна в колонке «старт»
    если машина не даёт столько потоков ОС: --app.hw14.swarm-max-platform-threads=N
        запускает пачками по N

Этап 6. StructuredTaskScope: группа виртуальных потоков под управлением скоупа
    подзадач 10000, обычная задержка 300 мс, срок 250 мс
    1) join() ждёт всю группу — все успешны (ShutdownOnFailure): подзадач 10000, с
        результатом 10000, с исключением 0, отменено 0, 319.2 мс, пик одновременных
        10000, итог: успех
       результатов после ожидания: 10000, все успешны: да, все подзадачи виртуальные: да
    2) одна падает через 100 мс — одна падает (ShutdownOnFailure): подзадач 10000, с
        результатом 0, с исключением 1, отменено 9999, 137.9 мс, пик одновременных
        10000, итог: исключение
    3) срок истёк — таймаут (ShutdownOnFailure): подзадач 10000, с результатом 0, с
        исключением 0, отменено 10000, 268.9 мс, пик одновременных 10000, итог: таймаут
    4) первый успешный источник — первый успех (ShutdownOnSuccess): подзадач 3, с
        результатом 1, с исключением 0, отменено 2, 152.8 мс, пик одновременных 3, итог:
        успех, ответ: источник №1 (ответ через 150 мс)
    сводка:
    сценарий      политика           подзадач  время, мс  успешно  сбоев  отменено  итог
    ------------  -----------------  --------  ---------  -------  -----  --------
          ----------
    все успешны   ShutdownOnFailure  10000     319.2      10000    0      0         успех
    одна падает   ShutdownOnFailure  10000     137.9      0        1      9999
          исключение
    таймаут       ShutdownOnFailure  10000     268.9      0        0      10000
          таймаут
    первый успех  ShutdownOnSuccess  3         152.8      1        0      2         успех
    обработка ошибок и отмены:
    все успешны: исключений нет; отменено 0 из 10000 (0 %), прогон 319.2 мс
    одна падает: IllegalStateException — «источник недоступен: подзадача №137»;
        отменено 9999 из 10000 (100 %), прогон 137.9 мс
    таймаут: TimeoutException — «группа не уложилась в 250 мс»; отменено 10000 из
        10000 (100 %), прогон 268.9 мс
    первый успех: исключений нет; отменено 2 из 3 (67 %), прогон 152.8 мс
    как это читать: скоуп — это блок, а не пул. Пока блок не закрыт, наружу выйти
        нельзя; отмена прерывает подзадачи и close() дожидается их завершения, поэтому
        «повисших» потоков не остаётся
    без отмены группа шла бы не меньше 400 мс (100 мс до сбоя + 300 мс паузы
        остальных), с отменой — 138 мс: прерывание 9999 подзадач тоже стоит времени, но
        до конца они не дорабатывали

Этап 7. Итог
    этап                исполнитель              время, мс  пик задач  итог
    ------------------  -----------------------  ---------  ---------
          ----------------------
    страницы            пул 16                   699.5      16         200 стр.
          загружены
    страницы            виртуальные              103.1      200        200 стр.
```

```
      загружены
pinning              synchronized             543.6      12         ограничен
      carrier'ами
pinning              ReentrantLock            61.1       96         carrier
      освобождается
масштабирование      пул 200                  2512.6     200        ждут в
      очереди пула
масштабирование      виртуальные              75.2       10000      все ждут
      одновременно
поток на задачу      new Thread().start       425.4      49         потоков ОС: +0
поток на задачу      Thread.ofVirtual().start 65.3       7983       потоков ОС: +0
StructuredTaskScope  ShutdownOnFailure        319.2      10000      успех
StructuredTaskScope  ShutdownOnFailure        137.9      10000      исключение
StructuredTaskScope  ShutdownOnFailure        268.9      10000      таймаут
StructuredTaskScope  ShutdownOnSuccess        152.8      3          успех

когда виртуальные потоки полезны: тысячи одновременных блокирующих операций
      (HTTP, БД, файлы) — потоков ОС остаётся столько же, сколько ядер
когда нет: CPU-нагрузка (сортировка, сжатие, вычисления) — виртуальный поток всё
      равно занимает carrier на всё время работы
чего избегать: synchronized вокруг блокирующих вызовов (pinning), пулы
      виртуальных потоков фиксированного размера, ThreadLocal на миллион задач
запросов к сайту за демонстрацию: 632, активных обработчиков сайта после этапов: 0

[tpm] Демонстрация завершена: все этапы пройдены, потерь задач нет, исполнители
      остановлены

живых платформенных потоков ДЗ 14 после завершения: []
виртуальные потоки в дамп не попадают: планировщик держит их вне
      Thread.getAllStackTraces(), поэтому отсутствие потерь проверяется счётчиками задач,
      а не списком потоков
```

`632` запроса — это 32 страницы прогрева плюс три полных прогона по 200 страниц (этап 1 и два прогона этапа 2).

4. Промежуточные результаты по этапам

Замеры на машине: 12 процессоров, 23 ГиБ ОЗУ, OpenJDK 21.0.13 (Temurin), Linux. Три последовательных прогона демонстрации:

Этап	Прогон 1	Прогон 2	Прогон 3	Вывод
200 страниц, виртуальные потоки	159.1 мс	154.3 мс	151.5 мс	все 200 одновременно, 12 carrier'ов
200 страниц, пул 16	699.5 мс	699.8 мс	693.3 мс	упирается в 16 потоков: 200 × 50 / 16 ≈ 625 мс
Ускорение (страницы, этап 2)	6.8×	6.2×	6.1×	стабильно в разы, не в проценты
Pinning: `synchronized`	543.6 мс, пик 12	483.9 мс, пик 12	484.2 мс, пик 12	8 кругов × 60 мс
Pinning: `ReentrantLock`	61.1 мс, пик 96	61.2 мс, пик 96	60.9 мс, пик 96	одна блокировка на всех

Этап	Прогон 1	Прогон 2	Прогон 3	Вывод
10 000 задач, пул 200	2512.6 мс	2517.8 мс	2518.1 мс	50 порций × 50 мс
10 000 задач, виртуальные	75.2 мс	75.8 мс	77.1 мс	одна задержка на всех
10 000 задач, `Thread.ofVirtual().start`	65.3 мс (старт 48.5)	31.2 мс (старт 7.4)	64.6 мс (старт 11.1)	пик задач 7983 / 7436 / 5894
10 000 задач, `new Thread().start`	425.4 мс (старт 424.3)	450.1 мс (старт 449.1)	465.7 мс (старт 464.6)	пик задач 49 / 49 / 47
CPU процесса (vt / pt)	520 / 1290 мс	230 / 1260 мс	480 / 1440 мс	платформенные дороже в 2.5–5 раз
STS: все успешны (10 000 × 300 мс)	319.2 мс	316.7 мс	321.0 мс	одна волна, пик 10 000
STS: падение одной	137.9 мс	142.3 мс	137.9 мс	9999 отменено, было бы ≥ 400 мс
STS: таймаут 250 мс	268.9 мс	268.3 мс	267.3 мс	10 000 отменено
STS: первый успех	152.8 мс	152.6 мс	152.5 мс	из 3 источников 2 отменены

Качественная картина устойчива, абсолютные миллисекунды плавают на 10–30 % — это ожидаемо для wall-clock замера без JMH. Разброс не влияет на выводы: разница между исполнителями измеряется разами. Сильнее всего плавает «старт» 10 000 виртуальных потоков (7–49 мс) и первый опыт pinning (543 мс против 484 мс в следующих прогонах) — это работа планировщика, JIT и фоновой нагрузки, а не свойство API.

5. Когда виртуальные потоки полезны, а когда нет

Полезны:

- тысячи одновременных блокирующих операций: HTTP, JDBC, файлы, ожидание ответа внешнего сервиса;
- «поток на запрос» в сервере: простое последовательное программирование без реактивных цепочек;
- задачи с редкими, но длинными паузами, где пул пришлось бы делать очень большим;
- там, где нужна отмена группы и гарантия «никто не остался работать», — `StructuredTaskScope`.

Не полезны:

- CPU-нагрузка: виртуальный поток занимает carrier-поток на всё время вычислений, и 10 000 виртуальных потоков на 12 ядрах будут работать не быстрее 12 платформенных;
- внутри `synchronized` вокруг блокирующего вызова (pinning) — эффект исчезает (на JDK 21);
- когда нужен ограниченный параллелизм: пул виртуальных потоков фиксированного размера бессмысленен, для ограничения числа одновременных операций нужен `Semaphore`, а не размер пула.

Чего избегать (pinning):

1. Не держать `synchronized` вокруг `Thread.sleep`, `InputStream.read`, `HttpClient.send`, `Future.get`, JDBC-вызовов. Заменить на `ReentrantLock` (или убрать блокировку из критической секции).
2. Не смешивать блокирующий IO и мониторы: критическая секция должна быть короткой и без ввода-вывода.
3. Следить за `jdk.virtualThreadScheduler.parallelism` и дампами: пришпиленные потоки видны в `jstack` как `VirtualThread[...] <pinned>` (в JDK 21; в JDK 24+ `synchronized` больше не пришпиливает — JEP 491).

4. Нативные вызовы (`JNI`) тоже пришпиливают поток — в JDK 21 это ограничение сохраняется.

Где структурная конкурентность, а не «просто виртуальные потоки»: `StructuredTaskScope` стоит брать там, где задача распадается на подзадачи с общим сроком и общей судьбой (собрать страницу из нескольких источников, опросить зеркала, разложить запрос на части). Он не заменяет пул для долгоживущих обработчиков: скоуп живёт один блок.

6. Автоматические проверки

```
cd app
mvn test -Dtest='ru.tsu.tpm.vacancyparser.hw14.**'
```

Результат: `Tests run: 32, Failures: 0, Errors: 0, Skipped: 0` (было 19, добавлено 13).

Тест	Что проверяет
`ThreadProbeTest` (2)	виртуальный поток виден как виртуальный и сообщает carrier; у платформенного carrier'а нет
`ConcurrencyMeterTest` (2)	пик одновременной работы, уменьшение счётчика при выходе, раздельный учёт виртуальных и платформенных исполнений
`VacancyPageServerTest` (3)	детерминированные заголовки и адреса, реальное применение задержки, освобождение порта, некорректные параметры
`PageClientTest` (2)	разбор `<title>` (атрибуты, регистр, сущности, вложенные теги), недоступный сайт — результат-сбой, а не исключение
`LoomBenchmarksTest` (3)	все задачи виртуальные и с результатом; пул ограничен своими потоками; отмена по таймауту, сбой задачи, пустой прогон, некорректные параметры
`PinningExperimentTest` (2)	`synchronized` ограничивает параллелизм числом carrier'ов и не даёт сменить carrier; `ReentrantLock` отпускает все задачи одновременно
`ThreadSwarmTest` (6, новые)	`Thread.ofVirtual().start` : 300 задач — 300 имён, все виртуальные; `new Thread().start` : 40 задач — 40 уникальных имён, все платформенные; ограничение одновременных потоков даёт ровно 4 пачки; каждая задача отдаёт своё имя в приёмник; `Thread.yield` вместо сна и пустой прогон; некорректные параметры
`StructuredExperimentsTest` (6, новые)	`join()` ждёт всю группу (200 подзадач, все с результатом, все виртуальные); падение одной отменяет остальные (9999 отменённых, `IllegalStateException` с номером подзадачи, прогон короче полной задержки); `joinUntil` + `shutdown` отменяет все 100 подзадач и укладывается в срок; первый успех отменяет дубли; разворачивание `ExecutionException` ; некорректные параметры
`LoomReportTest` (3)	выравнивание таблиц, сравнения, примеры заголовков, итоговая таблица, новые таблицы «поток на задачу» и «скоуп»
`Hw14PropertiesBindingTest` (2)	значения по умолчанию и переопределение всех `app.hw14.*` , включая новые параметры этапов 5–6
`LoomDemoConditionTest` (1)	демо создаётся только при `--app.hw=hw14`

Тесты проверяют **инварианты**, а не абсолютное время: миллисекунды зависят от машины, а «все задачи вернули результат», «работали виртуальные потоки», «при pinning пик не больше числа carrier'ов», «счётчики скоупа сходятся с числом подзадач» должны выполняться всегда. Поэтому прогон не флакует.

Сборка целиком: `mvn -pl app package` — успешно. Полный прогон всех домашних заданий: `Tests run: 577, Failures: 0, Errors: 0, Skipped: 0` (было 564 + 13 новых). hw01–hw13 и `finaltask` не затронуты.

7. Соответствие пунктам задания

Пункт задания	Где подтверждено	Отметка
1. 10 000 задач, каждая печатает имя потока и имитирует IO	этап 5: 10 000 имён в каждом прогоне, `Thread.sleep(1)`, пример вывода в 2.5.1	✅
2. Запуск через `Thread.ofVirtual().start(...)`	`ThreadSwarm.runVirtual`, этап 5: старт 48.5 мс, все 10 000 виртуальные	✅
3. Те же задачи через `new Thread(...).start()`	`ThreadSwarm.runPlatform`, этап 5: старт 424.3 мс, все 10 000 платформенные	✅
4. Замер времени `System.nanoTime()`, сравнение	`SwarmRun.startNanos/totalNanos`, таблица 2.5.2: 6.5× по времени и 2.5× по CPU	✅
5. `StructuredTaskScope`: группа, ожидание, исключения, отмена	этап 6: `join`, падение + отмена 9999, `joinUntil` + `shutdown`, `ShutdownOnSuccess`; разбор в 2.6.6	✅
6. Прогоны с разными количествами задач, время и ресурсы	раздел 2.5.3 (1000/5000/10 000), CPU процесса и куча в каждом прогоне, три прогона в разделе 4	✅
Отчёт: этапы, промежуточные результаты, объяснения, выводы	разделы 2–5	✅
Сравнение с платформенным пулом и разбор pinning (критерий приёма)	этапы 2–4: 6.8× на страницах, 33.4× на 10 000 задач, pinning 543.6→484 / 61.1 мс	✅

8. Как запустить и сдать

```
cd "Теория и практика многопоточности/app"

# Полная демонстрация (завершается с кодом 0); флаг preview уже передан
# плагином spring-boot-maven-plugin в форкаемую JVM
mvn spring-boot:run -Dspring-boot.run.arguments="--app.hw=hw14 --server.port=0"

# Готовый jar (обратите внимание на флаг preview)
mvn -DskipTests package
java --enable-preview -jar target/vacancy-parser.jar --app.hw=hw14 --server.port=0

# Больше страниц и другой размер пула
java --enable-preview -jar target/vacancy-parser.jar --app.hw=hw14 --server.port=0 \
  --app.hw14.pages=400 --app.hw14.fixed-pool-size=8

# Меньше задач в масштабировании (быстрее, но контраст слабее)
java --enable-preview -jar target/vacancy-parser.jar --app.hw=hw14 --server.port=0 \
  --app.hw14.scaling-tasks=2000 --app.hw14.scaling-delay-millis=20

# Этап 5 с другим числом задач и запуском платформенных пачками
java --enable-preview -jar target/vacancy-parser.jar --app.hw=hw14 --server.port=0 \
  --app.hw14.swarm-tasks=5000 --app.hw14.swarm-max-platform-threads=1000
```

```
# Этап 6 с другим сроком и падающей подзадачей
java --enable-preview -jar target/vacancy-parser.jar --app.hw=hw14 --server.port=0 \
  --app.hw14.structured-tasks=2000 --app.hw14.structured-timeout-millis=500
```

Параметры: `app.hw14.pages`, `app.hw14.page-delay-millis`, `app.hw14.fixed-pool-size`, `app.hw14.scaling-tasks`, `app.hw14.scaling-delay-millis`, `app.hw14.platform-pool-size`, `app.hw14.pinning-tasks`, `app.hw14.pinning-block-millis`, `app.hw14.request-timeout-millis`, `app.hw14.task-timeout-millis`, `app.hw14.sample-titles`, `app.hw14.swarm-tasks`, `app.hw14.swarm-sleep-millis`, `app.hw14.swarm-max-platform-threads`, `app.hw14.swarm-print-names`, `app.hw14.structured-tasks`, `app.hw14.structured-delay-millis`, `app.hw14.structured-fail-after-millis`, `app.hw14.structured-failing-task`, `app.hw14.structured-timeout-millis`, `app.hw14.structured-slow-delay-millis`, `app.hw14.structured-replica-delays-millis`.

Сдача: скопировать отчёт (`Отчёт_ДЗ_14.pdf` или этот README) в Google/Яндекс Документ, приложить исходники или ссылку на репозиторий, открыть доступ и прикрепить ссылку в личном кабинете.

9. Ограничения и честные оговорки

- **Preview-API и JDK 21.** `StructuredTaskScope` финальным не стал ни в 25, ни в 27 (проверено), поэтому проект собирается и запускается с `--enable-preview`, а jar работает только на JDK 21. Это осознанный выбор: на JDK 25 (JEP 491) ломается этап 3 (pinning) — два теста из 19 падают, и отчётные числа по pinning перестают быть верными. Все прочие классы проекта — обычный Java 21.
- **Замер — wall-clock, не JMH.** Нет прогрева в сотни итераций и статистики по десяткам прогонов; числа воспроизводимы качественно (разы), но не до миллисекунды. Для учебной демонстрации этого достаточно, для выбора библиотеки — нет.
- **Память — оценка.** `Runtime.totalMemory() - freeMemory()` включает ещё не собранный мусор; GC может отработать между замерами, поэтому «прирост» бывает отрицательным (в отчёте это помечено). Стеки платформенных потоков живут вне кучи, поэтому «0 МиБ по куче» у 10 000 потоков ОС — не экономия, а слепота замера: цена там в нативной памяти (~10 ГиБ адресного пространства). Точный замер требует `System.gc()` (искажает время) или JFR/async-profiler.
- **CPU — процесс в целом.** `OperatingSystemMXBean.getProcessCpuTime()` считает весь процесс, включая служебные потоки JVM и Spring; это не изолированный замер одного этапа. Поэтому в отчёте он приведён как «во сколько раз дороже», а не как абсолютная стоимость.
- **Счётчик платформенных потоков JVM общий на процесс и хранит исторический пик.** В него попадают и потоки Spring/Tomcat, и служебные потоки HTTP-клиента; если пик уже был достигнут раньше (например, в общем прогоне тестов), «прирост» равен нулю. Поэтому главный показатель — число имён потоков и пик задач из `ConcurrencyMeter`; счётчик JVM приведён как вспомогательная иллюстрация.
- **10 000 платформенных потоков — дорого и небесплатно для стенда.** Один прогон этапа 5 занимает ~0.5 с и ~10 ГиБ адресного пространства под стеки. На машинах с малым `ulimit -u` используйте `--app.hw14.swarm-max-platform-threads=N` (пачки) — логика и параметр описаны в 2.5.2.
- **Виртуальные потоки не видны в `Thread.getAllStackTraces()` и обычном дампе** — проверить «не осталось ли живых» списком потоков нельзя, поэтому отсутствие потерь проверяется счётчиками задач (200 из 200, 10 000 из 10 000) и `executor.shutdown()` / `close()` скоупа.
- **Задержка 50 мс и 200 страниц** — компромисс: меньше страниц — шумнее, больше — демонстрация удлиняется. 200 страниц × 16 потоков дают ~625 мс нижней границы для пула, что отлично от ~110 мс у виртуальных потоков с запасом.
- **`Thread.sleep(1)` в этапе 5 мал намеренно** (так в задании): из-за этого платформенные потоки не успевают накопиться и «пик задач» у них ~48, а не 10 000. Это ограничение сценария, а не замера; стоимость потока ОС всё равно видна в колонке «старт» и в CPU.
- **JDK 24+ изменит этап 3.** На JDK 24+ (JEP 491) `synchronized` больше не пришпиливает виртуальные потоки, и этап 3 покажет другие числа: оба прогона пройдут за ~60 мс. В отчёте указана версия JDK,

на которой сделаны замеры.